1) DIVIDE ET IMPERA

PROBLEME PROPUSE

1. Înmulțirea matricelor folosind algoritmul lui Strassen

Să se scrie un algoritm care calculează produsul a două matrice (C = A * B), folosind metoda divide-et-impera. Se va folosi următoarea abordare: o matrice n x n se poate scrie ca o matrice de dimensiune 2x2 ce conține ca elemente submatrice de dimensiuni n/2 x n/2. Pentru a înmulți două matrice 2x2 se folosesc doar 7 înmulțiri de submatrice, după cum este definit în continuare.

$$\begin{pmatrix} C11 & C12 \\ C21 & C22 \end{pmatrix} = \begin{pmatrix} A11 & A12 \\ A21 & A22 \end{pmatrix} * \begin{pmatrix} B11 & B12 \\ B21 & B22 \end{pmatrix}$$

Definim următoarele rezultate intermediare obținute prin înmulțirea submatricelor:

```
M1 = (A11 + A22)*(B11 + B22)
M2 = (A21 + A22)*B11
M3 = A11*(B12 - B22)
M4 = A22*(B21 - B11)

M5 = (A11 + A12)*B22
M6 = (A21 - A11)*(B11 + B12)
M7 = (A12 - A22)*(B21 + B22)
```

Folosind rezultatele intermediare putem obține submatricele care alcătuiesc rezultatul:

```
C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
```

2. Elementul median dintr-un vector

Fie o mulțime de numere reale cu n elemente reprezentată cu un vector v nesortat. Scrieți un algoritm de complexitate O(n) care să determine elementul median al mulțimii (echivalentul elementului de pe poziția n/2 din vectorul sortat, sau al k-lea (k =n/2) cel mai mic element din vector).

3. Subsecvența de sumă maximă

Fie un șir S = (s 1 , s 2 , ..., s N) de lungime n cu numere reale. O subsecvență a șirului este de forma: (s i , s i+1 , ..., s j) cu 1 <= i <= j <= n, iar suma subsecvenței este definită ca S ij = s i + s i+1 + ... + s j . Se cere să se determine subsecvența de sumă maximă folosind un algoritm divide-et-impera.

Exemplu: Pentru n = 7 și secvența: (5 -6 3 4 -2 3 -3), subsecvența de sumă maximă este:

(3 4 -2 3) a cărei sumă este 8.
Obs: Varianta cerută folosind o abordare de tip Divide et Impera are o complexitate O(n * log n), mai mare față de alternativa Greedy care este O(n).

INDICATII DE REZOLVARE

1. Înmulțirea matricelor folosind algoritmul lui Strassen

Inițial se vor adăuga linii și coloane cu elemente nule, până când cele două matrice vor fi pătratice, de dimensiune putere a lui 2, n = 2 k . Se partiționează matricele în blocuri de dimensiuni egale, ca în desenul de mai sus. Pentru a calcula cele 7 matrice intermediare M 1-7 , se apelează recursiv procedura de înmulțire (fiecare matrice M este produsul a altor două matrice de dimensiune mai mică n/2 = 2 k /2 = 2 k-1).
Complexitate: relația de recurență va fi T(n) = 7 * T(n/2) + n 2
Aplicând teorema Master, vom obține complexitatea algoritmului lui Strassen:
T(N) = O(n log 7) ≈ O(n 2 . 8)

2. Elementul median dintr-un vector

Ne propunem să rezolvăm generalizarea problemei – identificarea celui de-al k-lea cel mai mic element din vectorul inițial, urmând ca apelul final să folosească k = n/2). Această problemă se mai numește și problema selecției celui de-al k-lea element sau statistici de ordine.
Astfel, se pornește de la funcția de partiționare a algoritmului Quicksort. Fie q indexul returnat de această funcție. Dacă k = q atunci înseamnă că s-a găsit elementul căutat (v[k]). Dacă q > k, se va apela recursiv funcția de selecție pentru v[p...q-1], în caz contrar, se apelează pentru v[q+1...r], unde p și r reprezintă capetele șirului inițial.
Complexitate: O(n), dacă se folosește partiționare aleatoare.

3. Subsecvența de sumă maximă

Se divide șirul inițial în două subșiruri de lungime egală. Se calculează subsecvența de sumă maximă pentru fiecare din cele două subșiruri. Pentru a combina soluțiile rezultate, trebuie sa aflăm și subsecvența de sumă maximă ce are capetele în fiecare din cele două subșiruri. Astfel, se va alipi sufixul de sumă maximă din primul subșir cu prefixul de sumă maximă din cel de-al doilea subșir. Această operație de combinare a subsoluțiilor are complexitatea O(n) în cel mai defavorabil caz.
Complexitate: O(n * log n)

2) GREEDY

PROBLEME PROPUSE

1. Problema rucsacului (varianta continuă)

Un camion poate transporta T tone de material. Există n tipuri de materiale disponibile, fiecare caracterizat de greutatea G[i] disponibilă și de valoarea V[i] adusă de transportarea sa ($1 \leq i \leq n$). Să se decidă ce cantități din fiecare material vor fi transportate pentru a maximiza valoarea totală a produselor transportate de către camion.

2. Planificarea temelor de către studenți

Pe parcursul unui semestru, un student are de rezolvat n teme. Se cunoaște enunțul tuturor celor n teme de la începutul semestrului. Timpul de rezolvare pentru oricare dintre teme este de o săptămână și nu se poate lucra la mai multe teme în același timp. Pentru fiecare temă, se cunoaște și un termen limită d[i] (exprimat în săptămâni) și un punctaj w[i]. Nicio fracțiune din punctaj nu se mai poate obține după expirarea termenului limită al unei teme. Să se definească o planificare de realizare a temelor, în așa fel încât punctajul obținut la final să fie maxim.

3. Numere de tip palindrom

Se dorește construirea a două numere de tip palindrom (prin inversare – citirea de la dreapta la stânga – numărul rămâne la fel), primul de lungime $L > 0$, al doilea de lungime $l > 0$ astfel încât suma lor să fie minimă. Cele două numere de tip palindrom nu pot să înceapă cu cifra 0. Pentru construirea lor, se dau numărul L și numărul de apariții al fiecărei cifre în cele două numere (adică c[i] = nr. de apariții al cifrei i în primul numărul + nr. de apariții al cifrei i în al doilea număr, $0 \leq i \leq 9$).

INDICATII REZOLVARE

1. Problema rucsacului (varianta continuă)

Se calculează profitul obținut per unitatea de material tranportată V[i]/G[i] ($1 \leq i \leq n$) și se încarcă material în ordinea acestor valori. În plus, se poate observa că se obține o îmbunătățire a complexității – de la $O(n^2)$ la $O(n * \log n)$ – dacă valorile sunt sortate descrescător înainte de a face alegerile. Folosirea unei preprocesări simple (de exemplu, o sortare) în etapa inițială pentru a reduce complexitatea algoritmului este o proprietate a mai multor algoritmi construiți folosind tehnica Greedy.
Complexitate: $O(n * \log n) + O(n) = O(n * \log n)$

2. Planificarea temelor de către studenți

Se folosește un heap cu minim în vârf pentru a prioritiza temele în funcție de punctaj și de proximitatea termenului limită. Alternativ, pentru valori întregi și mici, se pot sorta temele după punctaje, în ordine descrescătoare și, în aceasta ordine, li se caută un slot de rezolvare, pornind de la termenul limită și căutând spre stânga.
Dacă în loc de maximizarea punctajului total obținut, se dorește maximizarea numărului de teme ce pot fi rezolvate, se sortează lista de intervale de rezolvare după termenul limită și se alege întotdeauna tema care se termină cel mai curând, în așa fel încât sa nu se suprapună cu nicio altă tema aleasă în prealabil.
Complexitate: $O(n * \log n)$

3. Numere de tip palindrom

Se observă că cifrele mai mici trebuie să ocupe poziții cât mai semnificative, ceea ce asigura că suma celor două numere să fie minimă. Pentru a realiza acest lucru se vor completa în paralel

cele două numere cu cifrele parcurse în ordine crescătoare stabilind în care din cele două numere se vor poziționa. De asemenea, trebuie avut în vedere ca niciunul din cele două numere să nu înceapă cu cifra 0.
Complexitate: O(L + l)

3) PROGRAMARE DINAMICA

PROBLEME PROPUSE

1. Problema rucsacului (varianta discretă)

Un camion poate transporta T tone de mobilă. Există n piese de mobilă, fiecare caracterizată de greutatea G[i] și de valoarea V[i] adusă de transportarea sa. Să se decidă ce piese de mobilă vor fi transportate pentru a aduce o valoare maximă (soluție optimă), știind ca nu avem voie să luăm bucăți dintr-o piesă de mobilă.

2. Aranjarea cărților în bibliotecă

Se dau n dulapuri aflate într-o bibliotecă, precum și numărul cărților din fiecare dulap. Se dorește mutarea dulapurilor (cu toate cărțile aferente) într-o nouă bibliotecă astfel încât primul dulap este mutat intotdeauna, iar celelalte dulapuri se mută doar dacă nu sunt așezate unul lângă celalalt în biblioteca inițială. De asemenea, se dorește aflarea soluției care maximizează numărul cărților din noua bibliotecă.

3. Parantezarea unei expresii booleene

Se dă o expresie booleană exprimată prin stringurile “true”, “false”, “and”, “or”, “xor”. Numărați modurile în care se pot așeza parantezele astfel încât rezultatul expresiei parantezate să fie “true”.
De exemplu, pentru expresia:
true and false xor true
există două modalitati de parantezare astfel încât rezultatul să fie “true”:
((true and false) xor true)
(true and (false xor true))

4. Înmulțirea optimă a unui lanț de matrici

Întrucât înmulțirea matricilor este asociativă, determinați modul optim de realizare al înmulțirilor unui șir A 1 ... A n de n matrici astfel încât numărul total de înmulțiri scalare să fie minimă.
Exemplu: Fie A 1 o matrice 10 × 30; A 2 - 30 × 5; A 3 - 5 × 60:
(A 1 * A 2) * A 3 = (10 × 30 × 5) + (10 × 5 × 60) = 1500 + 3000 = 4500 operații de înmulțire scalari
A 1 * (A 2 * A 3) = (30 × 5× 60) + (10 × 30 × 60) = 9000 + 18000 = 27000 operații de înmulțire scalari

INDICATII REZOLVARE

1. Problema rucsacului (varianta discretă)

Se folosește o matrice care se completează pe linii, iar fiecare linie modelează o lume în care camionul poate transporta 1 tonă, 2 tone, ... n tone. Putem modela o subproblemă ca fiind best[i][j] = valoarea maximă pe care o poate transporta un camion de tonaj maxim i folosind doar primele j piese de mobilă.
Relația de recurență este:
best[i][j] = max{best[i][j-1], best[i-G[j]] + V[j]}, dacă i - G[j] ≥ 0
best[i][j] = best[i][j-1] , dacă i - G[j] < 0
best[0][j] = 0; best[i][0] = 0
Soluția problemei va fi oferită de către best[T][n]. O observație interesantă este că această soluție poate fi aplicată doar dacă greutățile obiectelor sunt numere naturale, întrucât acestea intră în

definirea relației de recurență (vezi i - G[j]). În cazul general, problema rucsacului este NP-completă și nu acceptă o soluție polinomială.
Complexitate: O(n * T)

2. Aranjarea cărților în bibliotecă

Fie c_max[i] = numărul maxim de cărți ce pot fi transferate utilizând cărțile existente pe dulapurile de la 1 la i-2, astfel încât să fie selectat și dulapul i. Formula de recurență obținută este c_max(i) = carti(i) + maxim{c_max(k) unde k = 1, i-2}, ceea ce duce la o complexitate O(i) pentru pas și O(n 2) în total.
Totuși, relația de recurență de mai sus se poate optimiza observând că, la fiecare pas i, trebuie ales maximul dintre c_max(i-2) și c_max(i-3). Răspunsul este dat de maximul din vectorul c_max[0...n].
Complexitate: O(n)

3. Parantezarea unei expresii booleene

Se folosește o matrice pentru a stoca soluții parțiale pentru subprobleme, cu semnificația A[i][j] este numărul de parantezări asupra sub-expresiei formată din termenii pornind de la i și terminand cu j, astfel încât rezultatul sub-expresiei să fie true. Pozițiile se pot referi doar la elementele operanzi (de pe pozițiile pare: 0, 2, 4 etc.), nu și la operatori. Pentru simplitate, se poate utiliza o matrice separată care sa stocheze numărul de parantezări ale sub-expresiilor în asa fel încât rezultatul sa fie false. Cele două matrice vor fi simetrice; prin urmare, acestea pot fi completate pe subdiagonale, doar deasupra diagonalei principale, caz în care rezultatul final se va regăsi în colțul din dreapta-sus al matricei true.
Complexitate: O(n 2)

4. Înmulțirea optimă a unui lanț de matrici

Se folosește o matrice care se completează iterativ, peste diagonala principală, în care fiecare celulă modelează înmulțirea optimă a matricilor între indecșii i și j, precum și o structură auxiliară în care se memorează dimensiunea matricii rezultate din înmulțirea tuturor matricilor A i ... A j .
Ulterior, putem modela o subproblemă ca fiind best[i][j] = numărul minim de operații necesare pentru a înmulții matricilor A i ... A j . Relația de recurență este:
best[i][j] = min{best[i][k] + best[k+1][j] + costul înmulțirii matricii A i...k cu A k+1...j }
best[i][i] = 0;
Se poate observa ușor că aplicând relația de recurență pentru subproblema de dimensiune 1 (best[i][i+1]) obținem:
best[i][i+1] = costul înmulțirii matricii A i cu A i+1 = nr_linii(A i) * nr_coloane(A i) * nr_coloane(A i+1)
Soluția problemei inițiale va fi oferită de către best[0][n].
Complexitate: O(n 3)

4) BACKTRACKING SI OPTIMIZARI

PROBLEME PROPUSE

Sudoku

Jocul clasic rezolvat prin BKT la nivelul căruia aplicăm diverse optimizări este Sudoku. Astfel, avem o matrice 9 X 9 subdivizată în 9 sub-matrice identice de dimensiuni 3 X 3, denumite regiuni. Regula jocului este simplă: fiecare rând, coloană sau regiune nu trebuie să conţină decât o dată cifrele de la 1 la 9. Formulat altfel, fiecare ansamblu trebuie să conţină cifrele de la 1 la 9 o singură dată. Se cere să se modeleze această problemă folosind următoarele abordări:

- Backtracking simplu;
- BKT și o metodă prospectivă (preferabil predicție parțială sau completă);
- BKT și o euristică la alegere.

Pentru fiecare dintre soluțiile implementate se va analiza impactul la nivelul timpului total de execuție (care pot fi influențați și de alte procese din sistemul de operare, respectiv alocări și procese interne ale JVM, etc.), precum și al numărului de intrări în recursivitate.

INDICATII REZOLVARE

Varianta simplă de backtracking presupune introducerea tuturor valorilor posibile {1...9}, pe rând, în fiecare celulă liberă și verificarea consistenței atribuirii la nivel de linie, coloană și regiune. În cazul încălcării oricărei reguli, se încearcă o nouă valoare sau se revine în recursivitate, dacă nu mai există nicio variantă posibilă pentru variabila curentă. Prima optimizare presupune introducerea unei metode prospective care va filtra, la fiecare iterație, numărul de valori potențiale care pot fi introduse în anumite celule. Astfel, vor fi identificate și eliminate în prealabil potențialele inconsistențe la nivel de linie, coloană sau regiune (valori care nu pot fi utilizate în iterațiile ulterioare), iar dacă domeniul de valori al unei celule libere devine vid, este necesară revenirea din recursivitate. Suplimentar, se poate introduce și euristica de a reordona variabilele pornind de la cele mai restricționate (cu cele mai puține valori pontențial a fi introduse într-o anumită celulă), reducând astfel și mai mult numărul de intrări inutile în recursivitate.

5) MINIMAX

PROBLEME PROPUSE

X și O

Aplicația presupune aplicarea unor algoritmi de tip Minimax în scopul rezolvării jocului de X și O (tic-tac-toe). Astfel, se dorește implementarea algoritmul Mini/Nega-max astfel încât calculatorul să poată juca împotriva unui jucător uman. Adițional, trebuie extins algoritmul anterior prin utilizarea optimizării de tip tăietură alpha-beta (alpha-beta pruning).
Notă: acest joc, desi rezolvabil și prin metode mai puțin costisitoare din punct de vedere al efortului de calcul, însă este ales în scop didactic. Mai mult, varianta sa generalizată este mult mai complicată și exercițiul poate fi extins si pentru aceasta.

INDICATII REZOLVARE

Întrucât spațiul stărilor este limitat și numărul de mutări posibile scade pentru fiecare nivel deoarece matricea de stări este completată succesiv, implementarea poate fi utiliza adâncimea maximă (9), atât în cazul mini/nega-max normal, cât și în cazul aplicării tăierii alpha-beta. Funcția de evaluare va lua 3 valori posibile: +1 dacă jocul este câștigat de jucătorul curent, -1 dacă jocul este câștigat de oponent și 0 pentru toate stările în care nu se poate determina un câștigător. Pentru simplitate, se poate conveni din start dacă jucătorul uman este „X” sau „O”, și dacă acesta efectuează prima mutare.

6) PARCURGERE GRAFURI. SORTARE TOPOLOGICA.

PROBLEME PROPUSE

1. Trebuie să evadez! Nu mai rezist!

Se dă un tablou bidimensional reprezentând suprafața unei camere. Camera poate să conțină obstacole, reprezentate în matrice printr-o celulă de valoare 1, prin care nu se poate circula. Celulele cu valoarea 0 reprezintă elementele libere ale camerei prin care se poate trece. Dându-se un punct interior de start, să se decidă care este cel mai scurt drum până la o iesire. Ieșirile pot să fie doar pe marginea tabloului, și sunt simbolizate prin cifra 2. Să se afiseze și drumul până la una din cele mai apropiate ieșiri.

2. Cum ar trebui studiate optim materiile?

Se dă setul de mai jos de dependențe între materii (materia din stânga trebuie studiată într-un semestru anterior celei din dreapta). Să se precizeze o ordine de studiere a materiilor din curriculum.
Analiza Algoritmilor → Proiectarea Algoritmilor
Programarea Calculatoarelor → Proiectarea Algoritmilor
Structuri de Date → Proiectarea Algoritmilor
Proiectarea Algoritmilor → Algoritmi Paraleli și Distribuiti
Protocoale de Comunicatie → Algoritmi Paraleli și Distribuiti
Structuri de Date → Analiza Algoritmilor
Programarea Calculatoarelor → Protocoale de Comunicatie
Protocoale de Comunicatie → Rețele Locale
Algoritmi Paraleli și Distribuiți → Algoritmi și Prelucrări Paralele
Arhitectura Sistemelor de Calcul → Algoritmi și Prelucrări Paralele
Matematica 1 → Matematica 2
Matematica 2 → Matematici speciale
Matematica 1 → Fizică
Fizică → Electrotehnică
Fizică → Electronică Analogică
Electronică Analogică → Electronică DigitalăParcurgerea Grafurilor.

INDICATII REZOLVARE

1. Trebuie să evadez! Nu mai rezist!

Întrucât se dorește îndeplinirea criteriului de optimalitate se va aplica o parcurgere în lățime din poziția inițială, considerând totodată constrângerile impuse de celulele adiacente indisponibile (marcate cu 1). Labirintul poate fi modelat ca un graf, daca considerăm că pentru orice celulă (i, j), se poate construi ușor funcția succesori(i, j) care să întoarcă celulele adiacente în care se poate trece.

2. Cum ar trebui studiate optim materiile?

Se aplică o sortare topologică urmată de o alocare greedy pe semestre. Preferabil, se poate opta pentru includerea materiilor cu cele mai multe dependențe ulterioare în primele semestre. Pentru această evaluare se vor realiza parcurgeri suplimentare în adâncime pornind din fiecare materie și se vor prioritiza materiile în ordinea descrescătoare a adâncimii maxime.
Opțional, se poate realiza o combinație de Kahn și DFS pentru identificarea alocării optime.

7) APLICATII DFS

PROBLEME PROPUSE

Into the Matrix

Tocmai s-a lansat o noua rețea socială, dedicată exclusiv gamerilor. Întrucât toată lumea dorea să discute cu prietenii lor din comunitate într-un mod cât mai privat, comunicarea este unidirecțională prin canale dedicate. Astfel, atunci când un jucator intră prima dată pe un server, indică toate persoanele pe care le consideră prieteni și stabilește relațiile outbound. Ulterior, terțe persoane pot crea tuneluri permanente în sens invers pentru a permite comunicația bidirecțională între 2 noduri.

1) Identificați toate clanurile de pe un server. (Mesajele între oricare 2 membri dintr-un clan pot fi rutate prin canalele deschise)

2) Descoperiți ce relații există între clanuri. Care ar fi cel mai numeros clan dacă s-ar adauga o legatură artificială între oricare doi jucatori? Ulterior, pentru a spori ușurința utilizării, canalele de comunicație au devenit bi-direcționale și, pentru a evalua relațiile dintre jucători, dorim să identificăm următoarele elemente cheie din fiecare clan:

3) Care sunt jucătorii critici a căror eliminare ar duce la distrugerea comunicației în cadrul clanului?

4) Care canale între 2 jucători sunt critice la nivelul clanului?

INDICATII REZOLVARE

1) Graful este orientat și se aplica algoritmul lui Kosaraju sau Tarjan pentru identificarea componentelor tare conexe.

2) Varianta cea mai elegantă presupune contracția muchiilor (toate nodurile dintr-un CTC și muchiile aferente sunt contractate într-un singur nod), urmând ca muchia adăugată să creeze un ciclu în graful anterior și care să maximizeze numărul total de membri din CTC-uri incluse în ciclul nou creat. Soluția finală presupune adăugarea unei muchii înapoi între oricare dintre membrii celor două sub-comunități anterioare unificate prin muchia dintre CTC-uri.

3, 4). Jucătorii critici reprezintă punctele de articulație din graful neorientat, iar canalele critice sunt identificate sub formă de punți.

8) DRUMURI MINIME

PROBLEME PROPUSE

Zburând prin Europa

1. Andrei primește o hartă a Europei pe care sunt marcate toate zborurile posibile de la o localitate la alta, respectiv costurile acestora. Ca primă evaluare, Andrei își dorește să afle dacă două localități sunt conectate, indiferent de numărul de escale sau de linia aeriană.
2. Ulterior, Andrei își dorește să afle toate localitățile aflate pe drumul optim de la un oras la altul.
3. Întrucât Andrei se simte depășit de numărul foarte mare de localități de pe hartă, vă rugăm să identificați costul unei rute optime de la o singură localitate (cea în care se află Andrei) la oricare alta localitate.
4. Discutând cu un prieten, Andrei descoperă că pe anumite rute companiile oferă reduceri semnificative, clienții fideli primind chiar recompense financiare dacă aleg anumite rute. Determinați în aceste circumstanțe costul optim de la localitatea curentă la oricare alta localitate. În final, lui Andrei îi vine o idee ingenioasă și vă rugăm îl ajutați să descopere dacă exista posibilitatea de a specula recompensele liniilor aeriere pentru a circula la infinit între două localități, fără a plăti. În cazul identificării uneiastfel de rute, vă rugăm să o afișați.

INDICATII REZOLVARE

1) Pentru implementarea închiderii tranzitive se aplică algoritmul Floyd-Warshall puțin modificat: d[i, j] = (d[i, j] OR (d[i, k] AND d[k, j])).
2) Se aplică algoritmul Floyd-Warshall cu păstrarea părinților.
3) Se aplică algoritmul Dijkstra întrucât costurile muchiilor sunt pozitive.
4) În locul algoritmului Dijkstra trebuie aplicat algoritmul Bellman-Ford datorită costurilor negative, algoritm care permite totodată identificarea ciclurilor de cost negativ.

9) ARBORI MINIMI DE ACOPERIRE

PROBLEME PROPUSE

Cablare optimă în rețele de date

Prietenul nostru Andrei a fost asignat drept noul șef al departamentul de rețelistică al companiei Nuke Cola. Sediul companiei are arondate N sucursale, iar Andrei trebuie să conecteze asigure conectivitate între toate locațiile folosind o lungime minimă de fibră optică, lucru care duce implicit la reducerea costurilor totale. În cazul în care directorul general nu este de acord cu planul său, Andrei vrea să aibă un plan de backup, acesta reprezentând cea de-a doua cea mai bună soluție din punctul de vedere al lungimii totale de fibră optică utilizată. Vă rugăm să îl ajutați pe Andrei să găsească configurația optimă având drept intrare distanțele între locațiile care pot fi conectate direct una cu cealaltă.

INDICATII REZOLVARE

Cablare optimă în rețele de date

Inițial trebuie identificat arborele minim de acolerire (AMA) fosind fie algoritmul Prim, fie Kruskal. Ulterior, trebuie identificat cel de-al doilea arbore minim de acoperire (2AMA) utilizând următoarea construcție optimizată. Pentru fiecare muchie (u, v) $\notin$ AMA, se determină ciclul care

s-ar obține prin adăugarea muchiei (u,v) la AMA, urmând ca un candidat pentru 2AMA să se obțină prin eliminarea muchiei de cost maxim, diferită de (u, v), din acest ciclu nou creat. Astfel, în continuare este asigurată conectivitatea tuturor nodurilor (se elimină o muchie de cost maxim dintr-un ciclu), iar 2AMA este costul minim al tuturor arborilor construiți anterior pornind de la fiecare muchie care nu este în AMA-ul inițial. Pentru a optimiza construcția ciclurilor conținând muchia (u, v), se poate considera muchia de cost maxim din drumul de la u la v în AMA, cost care poate fi determinat cu ușurință folosind o parcurgere în lățime (BFS) în AMA.

10) FLUX MAXIM

PROBLEME PROPUSE

1. Networking

Pornind de la un graf orientat conex și două noduri u și v se cere să se determine următorele elemente:
a) Tăietura minimală a grafului afișată sub forma unei mulțimi de cardinal minim de muchii ce trebuie eliminate pentru a deconecta u și v.
b) Pentru topologia dată, afișați numărul maxim de drumuri disjuncte între u și v, dar și drumurile în sine. Două drumuri sunt considerate disjuncte dacă nu au nicio muchie în comun.

2. Generarea de grafuri

Se dă o listă de N noduri, pentru fiecare cunoscându-se gradul de intrare si de ieșire al acestora (In-degree și Out-degree). Implementați un algoritm care să construiască un graf orientat G cu N noduri, care să satisfacă constrângerile de grade (de intrare și de ieșire) date. Se poate construi maxim o muchie între oricare două noduri.

INDICATII REZOLVARE

1. Networking

a) Se aplică algoritmul Ford-Fulkerson, urmat de încă o parcurgere (BFS sau DFS) din nodul u pentru a detecta tăietura minimală. Această parcurgere va separa nodurile în 2 sub-mulțimi: o mulțime S1 cu noduri accesibile de u, respectiv mulțimea S2=V\S1 cu noduri inaccesibile care sunt blocate de muchiile saturate. Astfel, muchiile care trebuie eliminate sunt toate muchiile din graful G care au capătul stânga în S1 și capătul dreapta în S2.
b) Se aplică algoritmul Ford-Fulkerson adăugând fiecărei muchii o capacitate egală cu 1 și se afișează toate drumurile generate.

2. Generarea de grafuri

Se construiește un graf auxiliar pe care se aplică algoritmul Ford-Fulkerson. Acest graf are o sursă și o drenă virtuală și 2 straturi de N noduri (în total 2*N+2 noduri), iar arcele se construiesc astfel:
 Capacitățile muchiilor de la sursa virtuală la fiecare nod din primul strat de noduri este egal cu In-Degree; Flux Maxim 101
 Între cele 2 straturi de noduri se realizează un mesh/graf complet de muchii între nodurile primului strat cu capătul dreapta în al doilea strat (excluzând arcele de la nodul i din primul strat la nodul i din al doilea strat), toate de capacitate 1;
 Între fiecare nod din al doilea strat și drena virtuală se trasează arce de capacitate egal cu Out-Degree. Raportat la modelarea anterioare, se poate opta și pentru inversarea In-Degree cu Out-Degree, urmând ca muchiile saturat din mijloc să reflecte graful solicitat.

11) Algoritmi euristici. Algoritmul A*

PROBLEME PROPUSE

Problema comis voiajorului (Ciclu hamiltonian de cost minim)

Se dau n oraşe și rutele rutiere de interconectare dintre acestea. Se cere să se găsească un tur complet de lungime minimă (orașul de plecare și cel de întorcere sunt unul și același, iar toate celelalte orașe sunt străbătute o singură dată). Pentru fiecare dintre variantele propuse de rezolvare se va evalua numărul de stări explorate și se va interpreta evoluția acestora raportat la soluția și euristica propusă.

- Să se rezolve problema folosind o abordare de tip Best-First.
- Să se rezolve problema folosind o euristică simplă A*, admisibilă, definită în raport cu o singură localitate neexplorată în cadrul stării curente.
- Rafinați soluția de la punctul b), propunând o euristică mai informată prin evaluarea întregului orizont neexplorat (variabilele = localitățile care nu au fost deja parcurse în starea curentă).

INDICATII REZOLVARE

În prima etapă se consideră minimul distanței deja acoperite de localitățile selectate. Ulterior, pe lângă funcția g(x) definită anterior, se va considera h(x) = distanța către cel mai apropiat oraș neparcurs. Euristica cea mai bună presupune utilizarea, în locul funcției h(x) definită anterior, a costul arborelui minim de acoperire (AMA) aferent orașelor neparcurse.

12) ALGORITMI ALEATORI

PROBLEME PROPUSE

k-Means

Fie n puncte într-un spațiu bi-dimensional. Se dorește o grupare a acestora în k clustere – un cluster este un grup de puncte situate într-o vecinătate spațială care să maximizeze distanța internă (coeziune intra-cluster) și să asigure o cuplare slabă inter-clustere.

De exemplu, pentru setul de intrare:

X	2	3	2	3	3	3	4	5	8	8	9	10	10	10
Y	9	10	5	4	3	2	1	1	2	3	4	5	6	7

Se poate observa o grupare naturală în 3 clustere:

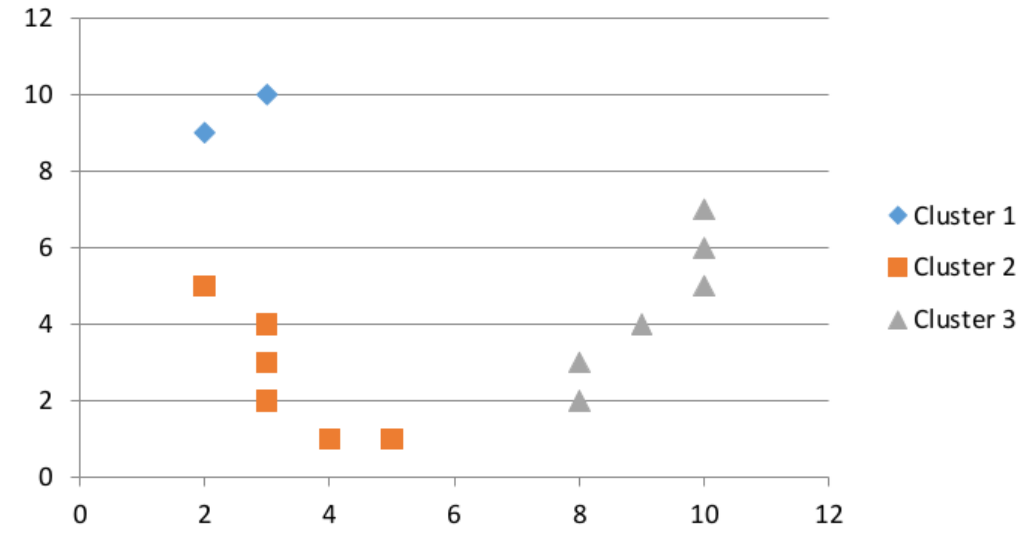


Figura 12.1 – Exemplu de clustering pentru setul de intrare format din 14 puncte

Pașii algoritmului sunt următorii (pentru un n și un k predefinite):
 Se selectează k puncte random din spațiu care vor fi centroizii inițiali ai clusterelor.
 Se efectuează iterativ următorii pași, până când atribuirile fiecărui punct la un cluster rămân neschimbate între doi pași consecutivi:
 Asignarea fiecărui punct unui cluster în mod greedy pentru fiecare punct în parte (acel centroid care minimizează distanța euclidiană către centroizii din pasul curent este minimă);
 Recalcularea centroidului drept media aritmetică a coordonatelor punctelor asignate (se face acest lucru pentru fiecare coordonată). Pentru un set de date și un k stabilit se vor determina clusterele aferente. Se va implementa și o optimizare a selecției inițiale de puncte pornind de la principiul că acestea ar trebui să fie geometric cât mai dispersate (se dorește maximizarea distanței între centroizii inițiali).

INDICATII REZOLVARE

Se implementează cele 2 etape specifice algoritmilor de tip Expectation Maximization:
 Asignarea fiecărui nod la unul dintre cele k clustere (distanța minimă euclidiană către centroizii din pasul curent este minimă);

 Recalcularea centroidului drept media aritmetică a coordonatelor
punctelor asignate fiecărui cluster. Pentru o asignare inițială mai eficientă, se pornește cu un punct random n 0. Ulterior, fiecare punct n i , cu $1 \leq i \leq k$, se alege pe considerentul că suma distanțelor până la toate punctele anterioare este maximă. Astfel, pentru n 1 se urmărește maximizarea distanței la n 0 , pentru n 2 suma distanțelor la n 1 și n 0, etc. În final, punctele n 1...k (n 0 este exclus) reprezintă alocările inițiale care asigură o dispersie cât mai mare a centroizilor.